

[Guidelines](#)

[Objective](#)

[Contribution Declaration](#)

[Steps](#)

[Important Dates](#)

[Recommendations](#)

[What happens after the publication of the post?](#)

[RecSys- Capstone SuperBlog](#)

[References](#)

[Item2Vec: Neural Item Embedding for Collaborative Filtering](#)

[Papers on applications](#)

[Introduction](#)

[Breakout 2](#)

[Introduction and Background](#)

[The BERT4Rec Model Architecture](#)

[The Transformer Layer](#)

[What is the difference between BERT4Rec and the model introduced in \(Vaswani, 2017\)?](#)

[Multi-Head Attention](#)

[Feed-Forward block](#)

[Positional Encoding](#)

[Model Training Specifics](#)

[Experimental results and conclusion](#)

[Works Cited](#)

[Breakout 3](#)

[Introduction: Why should you read this?](#)

[Key Contributions](#)

[What is SARSA and Q-learning?](#)

[Markov Decision Process \(MDP\) Model for Slate Recommendation](#)

[What is SlateQ Decomposition?](#)

[How to find the slate with maximum long-term value?](#)

[Adapting Myopic Systems into Long-Term Systems](#)

[State Space](#)

[Generalization across users](#)

[User Response Modeling](#)

[Infrastructure](#)

[Results](#)

[Future Directions](#)

[Breakout 4](#)

[Breakout 5](#)

[Summary](#)

[Introduction](#)

[Problem Formulation and Approach Analysis](#)

[Problem Formulation](#)

[ALS with Weighted-Regularization Approach](#)

[Parallel ALS with Weighted- \$\lambda\$ -Regularization](#)

[Performance for the Netflix Prize Problem](#)

[Post-processing](#)

[Experimental Results for ALS](#)

[Other Methods and Linear Blending](#)

[5 Related Work](#)

[Breakout 6](#)

[Breakout 7](#)

Guidelines

This blog post is the collective work of the participants of the “Recommender Systems” workshop. This post serves as a proof of work, and covers some of the concepts covered in the workshop in addition to advanced concepts pursued by students. The blog post will be shared on the AISC Blog, and other major ML/DS related outlets.

Objective

The objective of this post is to demonstrate an understanding of the concepts learned in the Workshop. Teams will be tasked with summarizing a selected RL paper and providing additional insight into the findings and concepts discussed.

Contribution Declaration

Each team will provide contribution acknowledgement to its members for participating in the blog post writing and idea generation.

Steps

1. Refer to the **References** section and select one of the suggested papers. Given the topics discussed in the workshop, you should be familiar with the concepts used in each of the papers.
2. Please **list the names of your group and team members** that contributed to the development at the top of each section.
3. Work with your **breakout team** to go through the paper, and support each other to make sure everyone understands it well; you can divide and conquer by reading/researching different parts and explaining to each other
4. Come up with a **work plan for each member** of the team to contribute something (the updated version of this should later turn into the “contribution declaration”)
5. **Collaborate and draft** your learnings into a section in this article; the alternative is to create a video about what you learned
6. AISC blog editors will provide light **technical and language feedback** on the post prior to publication on the AISC Blog.
7. AISC blog editors will provide guidance **before** submitting the blog post to other mediums such as Towards Data Science (Medium.com) and KDNuggets.

Important Dates

There are 2 deliverable options based on your available time commitment. Please select the option that suits your team members.

1. **Nov 4: Select the paper** you want to work on

2. **Nov 11:** Write a **qualitative summary of what you learned** about the paper you selected. Please provide enough technical details to demonstrate your understanding
3. **Nov 25: Reproduce the results of the paper** by going through the code, and rerunning it on the given data set. This should link back to your github repo, or any other pages showing your results/code

If you've completed both exercises and would like an additional challenge, please contact us at events@ai.science and we can provide guidance to find an extension that could result in some sort of research publication, or a simple application that you can use as part of your portfolio

Recommendations

- You may pick a paper that is not listed here as long as you convince your team; please add that paper to the **References** section
- We generally prefer that you don't work alone, but if you have very good reasons for it, we might consider it
- You are encouraged to read other sections and provide constructive feedback in the form of comments, but please do not alter them
- If you claim a section to write, then you need to deliver by the due dates above. If you miss the deadline, the post will be published without your section
- Breakout teams are combinations of in-person and online audience, and it's everyone's responsibility to make sure that all team members are engaged and informed about plans. You can use the slack channel for your communication as much as you want, but also can arrange for video calls etc.
- If you use any other resources, add their information to the **References** section, but make sure you don't modify the existing **References**

What happens after the publication of the post?

We will provide you exposure through our channels including reposting on major ML/DS blogs, inviting you to speak at our community events, etc. **In rare cases a small number of ambitious participants who finish both milestones will be chosen to continue working with us on research projects with the purpose of scientific publications, or generating IP.**
[SCROLL TO THE NEXT PAGE]

RecSys- Capstone SuperBlog

References

Papers on algos and concepts:

- [Large-scale Parallel Collaborative Filtering for the Netflix Prize](#)
- [Variational Autoencoders for Collaborative Filtering](#)
- [node2vec: Scalable Feature Learning for Networks](#)
- [Amazon's item-to-item collaborative filtering](#)
- [An MDP-based recommender system](#)

- [Item2Vec: Neural Item Embedding for Collaborative Filtering](#)

Papers on applications

- [Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches](#)

Papers on NLP being used in RecSys:

- [BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer](#)
- [Session-based recommendations with recurrent neural networks](#)
- [Pre-training of Context-aware Item Representation for Next Basket Recommendation](#)

Introduction

[one paragraph summary of session 1]

[one paragraph summary of session 2]

[one paragraph summary of session 3]

TODO [one paragraph outlining the post; has to be written once the sections are filled]

Breakout 2

Title of the paper: BERT4Rec: Bidirectional sequential recommendations

Team members: Robert Hensley, Madhur Kanjolia, Hari Ravindran, Tryambak Kaushik

Contributions: The sections on model architecture and model training specifics were co-written by Robert Hensley and Tryambak Kaushik, experimental results and conclusions were written by Madhur Kanjolia, while the introduction/background and final editing/formatting of the report was done by Hari Ravindran.

Introduction and Background

The following report will provide a summary of work done in (Sun, 2019), an electronic preprint outlining recent research conducted by machine learning scientists belonging to the Recommendation Team at Alibaba Group in the context of recommender systems.

The essential crux of the hypothesis put forward in (Sun, 2019) is that sequential recommender systems which employ neural networks (such as Recurrent Neural Networks) and unidirectional sequential models to make recommendations based on a user's historical interactions, while effective and prevalent in a lot of cutting-edge research concerning recommendation systems, are not nearly sufficient enough to capture the multifaceted and complicated pattern of user behaviour sequences. Instead, inspired by the success of BERT, a state-of-the-art NLP model first introduced in (Devlin, 2019), in text understanding, (Sun, 2019) proposes the use of deep bidirectional self-attention models to sequential recommendations.

Before we dive into the nuts and bolts of the approach undertaken in (Sun, 2019), we shall first set the stage for what is to follow by outlining some recent and not-so-recent work that serve as precursors to the work in question.

Traditionally, all recommender systems were (and to a great extent, still are) classified into three distinct categories: collaborative filtering, content-based and hybrid recommender systems, each referring to a certain distinct way of extracting information from user-item interactions. The interested reader can refer to (Aggarwal, 2016) for more details regarding this taxonomy. Classical methods used in the pursuit of crafting these systems relied primarily on matrix factorization techniques together with k-nn (nearest neighbour)-based recommender systems to glean insights from available data.

Initial research in recommender systems, which lurched from one incremental improvement to another, primarily lay ensconced within the realm of academia and corporate R&D labs until Netflix publicly announced the Netflix Prize, a competition that ran from 2006 to 2009 with a million dollar payout to a team capable of producing a model capable of beating Netflix's own recommendation system by a margin of 10%. Each team participating in the competition, in turn, had access to an anonymized dataset of 100 million movie ratings released by Netflix at the start of the competition. The Netflix Prize served as a shot in the arm for research in this nascent field, leading to an explosion in the variety of methods and algorithms used to tackle the thorny problem of making accurate recommendations.

One such approach was outlined in (Salakhutdinov, 2007), wherein a two-layer Restricted Boltzmann Machine (RBM) was used for collaborative filtering, which was probably the first recommendation model to be built on neural networks. While the approach itself didn't win the Netflix prize, we mention it specifically because it served as a stepping stone for the advent of neural networks and deep learning in the context of recommender systems. The current work, (Sun, 2019), is one such example of work that uses an approach rooted in deep learning to deal with recommendation systems.

Following the taxonomy of deep-learning based recommendation systems outlined in (Zhang S., 2019), we can classify recommendation models into roughly two types:

- Recommendation systems with neural building blocks
- Recommendation systems with deep hybrid models

With a sophisticated architecture using a variety of deep learning techniques in the pursuit of generating better recommendations, the current work clearly falls within the latter category.

Before delving into the minutiae of the model under discussion, its workings can be summarized as follows: signals that combine the sequential patterns of user behaviour together with the temporal dynamics of user-product interactions are used to generate product/item recommendations. The main difference (or innovation even) in the approach being discussed here lies in the use of BERT (Bidirectional Encoder Representations from Transformer), a state-of-the-art NLP model released by the Google AI (Language) team in 2018, within the realm of recommender systems.

The BERT4Rec Model Architecture

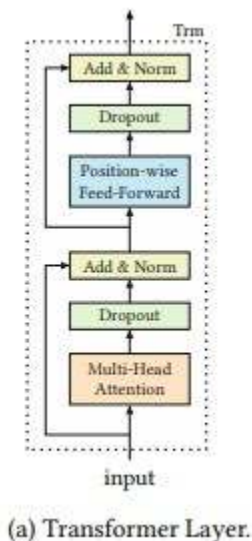
As noted by the authors in (Sun, 2019), an integral component of BERT4Rec is a self-attention layer called the 'Transformer Layer'. Attention mechanisms capable of filtering out uninformative features from raw inputs have long (relatively speaking) been used with some success in tasks such as computer vision, NLP, and speech recognition. (Ouyang Y., 2014) was one of the earliest works to propose the use of attention mechanisms in collaborative filtering by introducing a two-level attention mechanism to a latent factor model.

The Transformer Layer

In a groundbreaking 2017 paper (Vaswani, 2017), a team of Google researchers suggested that 'Attention Is All You Need'. As noted in Chapter 16 of (Geron, 2019), the researchers 'created an architecture called the Transformer, which significantly improved the state of the art in neural machine translation without using any recurrent or convolutional layers, just attention

mechanisms (plus embedding layers, dense layers, normalization layers, and a few other bits and pieces). As an extra bonus, this architecture was also much faster to train and easier to parallelize, so they managed to train it at a fraction of the time and cost of the previous state-of-the-art models’.

The basic structure of the Transformer Layer introduced in (Vaswani, 2017) is an attention mechanism block, followed by a feed-forward block (shown in the figure below).



Each of the two blocks applies dropout to its output (also called a ‘sublayer’), which is then added to the original input of that block, to which Layer Normalization is finally applied – making it ready to feed into the next layer as the final output. Simply put, for each block, using a form reminiscent of Keras’ functional API, we have the following chain of operations, with the output of one acting as input to the other:

$$final_output = Layer_Norm(input + Dropout(SubLayer(input)))$$

Transformer layer and multi-head attention were successfully used by Devlin et al (2018) for bi-directional learning and better the existing natural language models. Sun (2019) extend the concepts of Transformer architecture for recommendation systems.

What is the difference between BERT4Rec and BERT (Devlin et al, 2018) the model introduced in (Vaswani, 2017)?

BERT4Rec formulates the recommendation system as a language modeling task and simply uses the encoder self-attention structure, similar to BERT, to generate latent layers. However, BERT4Rec discards , and discardsing the decoder part of BERT because decoding the latent layer to a target language (or space) is not required by a recommender system. The latent space, thus generated, is compared with the ground truth to minimize loss function and update

the model parameters. BERT4Rec also differs from BERT in data pre-processing. BERT4Rec Secondly, authors use , and trains using a Cloze task to randomly mask several items in the input sequence, also popularly known as Cloze task. Hence, BERT4Rec must now predict several items of a sequence and not just the last item unlike a language model. This prevents overfitting and also provides more datasets for model to train as model is no longer expected to predict only the last item.

Cloze task – which randomly masks portions of the input in order to coax the model into bidirectional representations, instead of the unidirectional ‘predict next’ paradigm that would be seen in the decoder.

A few more differences worth noting:

- BERT4Rec has more parameters than the original encoder
- It uses GELU activations for the feed-forward layers instead of ReLU activations
- At test time, BERT4Rec simply appends a mask to the end of the input and thus to simulate a Cloze mask prediction (introduced in (Taylor, 1953)) for bidirectional properties - namely a non-rigid order of the time series - while effectively forcing the non-rigid expected output back to a rigid “predict next” paradigm.

Multi-Head Attention

AAs noted earlier, the attention mechanism in machine learning, similar to social human behavior, refers to focusing attention on relevant features or parameters and consequently learning data inter-dependence more efficiently, in a manner very similar to social human behavior. In the case of self-attention, this is achieved by using hidden features (from the hidden layers of deep learning) of the primary variable input data itself and does not explicitly requires data labels. Hence, self-attention in machine learning is a self-sufficient technique. and does not require data labels for learning features requires minimal explicit feature extraction from the raw input data.

BERT4Rec uses ‘Multi-head attention’, a popular self-attention technique from ‘Multi-head self-attention’ of The attention block in BERT4Rec is the same as in (Vaswani et al (, 2017) to model recommender systems. The attention mechanism used is known as ‘multi-head self-attention’. The technique is named ‘multi-head’ as it uses several ‘heads’ to capture different aspects of the sequence. These are weighted summed to develop the complete sequence representation. The keys and queries are used to generate the ‘attention’ along the sequence and end up being the same as they ‘self-attend’. The multi-headed self-attention, thus forms takes in the Query and Keys from the sequence block’s input ,to form Query and Key matrices that are compared them to each other and normalizes them by model size and the number of heads. Theis forms resulting a similarity matrix which is then compared to the Value matrix (all comparisons are done using matrix multiplication), providing us with the sublayer output for the attention block.

Feed-Forward block

One of the most notable changes to the original encoder used in (Vaswani, 2017) can be seen in the Feed-Forward block - the ReLU nonlinearity is replaced with a GELU nonlinearity. The GELU activation function is non-convex, non-monotonic, non-linear in positive domain and

consequently, is thus possibly able to better approximate the complex functions than ReLUs or ELUs (Hendrycks, 2016).

In (Sun, 2019), the concatenation of attention head outputs is projected on GELU to generate the feed-forward block as follows:

$$ff_out = GELU(input * W1 + b1) * W2 + b2$$

Dropout is then used on the output, added to the original input of the block, before applying layer normalization.

$$final_output = Layer_Norm(input + Dropout(ff_out(input)))$$

Positional Encoding

As a parallelized process, the Transformer is often unable to keep track of word sequence in the input sentence. Thus, a positional encoding is frequently implemented with the Transformer layer. Positional encoding, as the name suggests, involves generating an encoded representation of the relative position of the words. Positional representation is subsequently combined with the transformer encoding to complete the architecture. Unlike the classical self-attention transformer introduced in (Vaswani, 2017), BERT4Rec does not use a constant positional encoder and instead defines a learnable positional encoder to improve model performance.

The final output layer of the BERT4Rec model transforms the hidden representations into item probabilities. It is a two-layer feed-forward network which accepts hidden representations (ht) and item embeddings (E):

$$P(v) = softmax(GELU(ht * Wp + bp) * E_transpose + bo)$$

The learnable projection is 'Wp' with its bias 'bp'. The learned projection of the hidden representations is compared to the items set (E) to yield the item probabilities.

Model Training Specifics

For training, the model is made to predict the masked portions of the sequence - masks in this task are actual <mask> tokens. In any given sequence multiple items are masked. Below is an example mask of a items sequence vector:

Input: $[v_1, v_2, v_3, v_4, v_5]$ $\xrightarrow{\text{randomly mask}}$ $[v_1, [\text{mask}]_1, v_3, [\text{mask}]_2, v_5]$ Labels: $[\text{mask}]_1 = v_2, [\text{mask}]_2 = v_4$
--

The model then optimizes on a negative log likelihood for each item that was masked:

$$\mathcal{L} = \frac{1}{|S_u^m|} \sum_{v_m \in S_u^m} -\log P(v_m = v_m^* | S_u')$$

Here we have S'_u as the masked version of the sequence, v_m^* as the specific true value, and S_u^m as the set of true values for the sequence S_u .

Experimental results and conclusion

In (Sun, 2019), the BERT4Rec model was evaluated on four real world datasets, namely MovieLens (20M and 1M), Steam and Amazon Beauty product review data. The summary statistics related to these 4 datasets are shown below in Table 1 from (Sun, 2019):

Table 1: Statistics of datasets.

Datasets	#users	#items	#actions	Avg. length	Density
Beauty	40,226	54,542	0.35m	8.8	0.02%
Steam	281,428	13,044	3.5m	12.4	0.10%
ML-1m	6040	3416	1.0m	163.5	4.79%
ML-20m	138,493	26,744	20m	144.4	0.54%

All the datasets were preprocessed to convert numeric ratings to an implicit feedback of one. Interaction records were grouped by user, and then sorted by timestamp to build interaction sequences. Finally, only users with at least 5 feedback scores were kept for training purposes.

For evaluation, the last item was considered as part of test data, and the penultimate item as part of validation; the model was trained on the remaining items for any user. For easy and fair evaluation, each ground truth item in the test group is paired with 100 randomly selected negative items that the user did not interact with, sampled by their popularity for reliable and representative sampling. The metrics used for evaluation were: hit rate, MRR (mean reciprocal rank), and NDCG (Normalized discounted cumulative gain).

Some of the baseline models compared against the performance of the model were: POP (where items are ranked according to the number of transactions), BPR-MF (introduced in (Rendle S., 2012)), GRU4Rec (Hidasi B., 2015), and SASRec (Kang W.C., 2018).

As tabulated in (Sun, 2019), the performance results of BERT4Rec are outlined below. The last column of the table demonstrates the improvement of BERT4Rec relative to the best baseline.

Table 2: Performance comparison of different methods on next-item prediction. Bold scores are the best in each row, while underlined scores are the second best. Improvements over baselines are statistically significant with $p < 0.01$.

Datasets	Metric	POP	BPR-MF	NCF	FPMC	GRU4Rec	GRU4Rec [†]	Caser	SASRec	BERT4Rec	Improv.
Beauty	HR@1	0.0077	0.0415	0.0407	0.0435	0.0402	0.0551	0.0475	<u>0.0906</u>	0.0953	5.19%
	HR@5	0.0392	0.1209	0.1305	0.1387	0.1315	0.1781	0.1625	<u>0.1934</u>	0.2207	14.12%
	HR@10	0.0762	0.1992	0.2142	0.2401	0.2343	0.2654	0.2590	<u>0.2653</u>	0.3025	14.02%
	NDCG@5	0.0230	0.0814	0.0855	0.0902	0.0812	0.1172	0.1050	<u>0.1436</u>	0.1599	11.35%
	NDCG@10	0.0349	0.1064	0.1124	0.1211	0.1074	0.1453	0.1360	<u>0.1633</u>	0.1862	14.02%
	MRR	0.0437	0.1006	0.1043	0.1056	0.1023	0.1299	0.1205	<u>0.1536</u>	0.1701	10.74%
Steam	HR@1	0.0159	0.0314	0.0246	0.0358	0.0574	0.0812	0.0495	<u>0.0885</u>	0.0957	8.14%
	HR@5	0.0805	0.1177	0.1203	0.1517	0.2171	0.2391	0.1766	<u>0.2559</u>	0.2710	5.90%
	HR@10	0.1389	0.1993	0.2169	0.2551	0.3313	0.3594	0.2870	<u>0.3783</u>	0.4013	6.08%
	NDCG@5	0.0477	0.0744	0.0717	0.0945	0.1370	0.1613	0.1131	<u>0.1727</u>	0.1842	6.66%
	NDCG@10	0.0665	0.1005	0.1026	0.1283	0.1802	0.2053	0.1484	<u>0.2147</u>	0.2261	5.31%
	MRR	0.0669	0.0942	0.0932	0.1139	0.1420	0.1757	0.1305	<u>0.1874</u>	0.1949	4.00%
ML-1m	HR@1	0.0141	0.0914	0.0397	0.1386	0.1583	0.2092	0.2194	<u>0.2351</u>	0.2863	21.78%
	HR@5	0.0715	0.2866	0.1932	0.4297	0.4673	0.5103	0.5353	<u>0.5434</u>	0.5876	8.13%
	HR@10	0.1358	0.4301	0.3477	0.5946	0.6207	0.6351	0.6692	<u>0.6629</u>	0.6970	4.15%
	NDCG@5	0.0416	0.1903	0.1146	0.2885	0.3196	0.3705	0.3832	<u>0.3980</u>	0.4454	11.91%
	NDCG@10	0.0621	0.2365	0.1640	0.3439	0.3627	0.4064	0.4268	<u>0.4368</u>	0.4818	10.32%
	MRR	0.0627	0.2009	0.1358	0.2891	0.3041	0.3462	0.3648	<u>0.3790</u>	0.4254	12.24%
ML-20m	HR@1	0.0221	0.0553	0.0231	0.1079	0.1459	0.2021	0.1232	<u>0.2544</u>	0.3440	35.22%
	HR@5	0.0805	0.2128	0.1358	0.3601	0.4657	0.5118	0.3804	<u>0.5727</u>	0.6323	10.41%
	HR@10	0.1378	0.3538	0.2922	0.5201	0.5844	0.6524	0.5427	<u>0.7136</u>	0.7473	4.72%
	NDCG@5	0.0511	0.1332	0.0771	0.2239	0.3090	0.3630	0.2538	<u>0.4208</u>	0.4967	18.04%
	NDCG@10	0.0695	0.1786	0.1271	0.2895	0.3637	0.4087	0.3062	<u>0.4665</u>	0.5340	14.47%
	MRR	0.0709	0.1503	0.1072	0.2273	0.2967	0.3476	0.2529	<u>0.4026</u>	0.4785	18.85%

We also note some other important findings from this detailed study:

- The gain in performance over best baseline can be attributed to both the bidirectional representation of the sequential recommendation process and the Cloze objective in BERT4Rec (Also shown in below Table 3 from (Sun, 2019)).

Table 3: Analysis on bidirection and Cloze with $d = 256$.

Model	Beauty			ML-1m		
	HR@10	NDCG@10	MRR	HR@10	NDCG@10	MRR
SASRec	0.2653	0.1633	0.1536	0.6629	0.4368	0.3790
BERT4Rec (1 mask)	0.2940	0.1769	0.1618	0.6869	0.4696	0.4127
BERT4Rec	0.3025	0.1862	0.1701	0.6970	0.4818	0.4254

- The bidirectional model outperforms the state-of-the art unidirectional baseline model from (Kang W.C., 2018)
- BERT4Rec outperforms all other baselines on all other datasets with a relatively small hidden dimensionality (Figure 3 from (Sun, 2019)). The performance of the model converges as hidden dimensionality(d) increases (d values in graph considered for comparative study : 16, 32, 64, 128, 256). We also note that larger dimensionality does not necessarily lead to better performance, especially on sparse datasets. Self-attention based methods (SASRec and BERT4Rec) achieve superior performance on all datasets

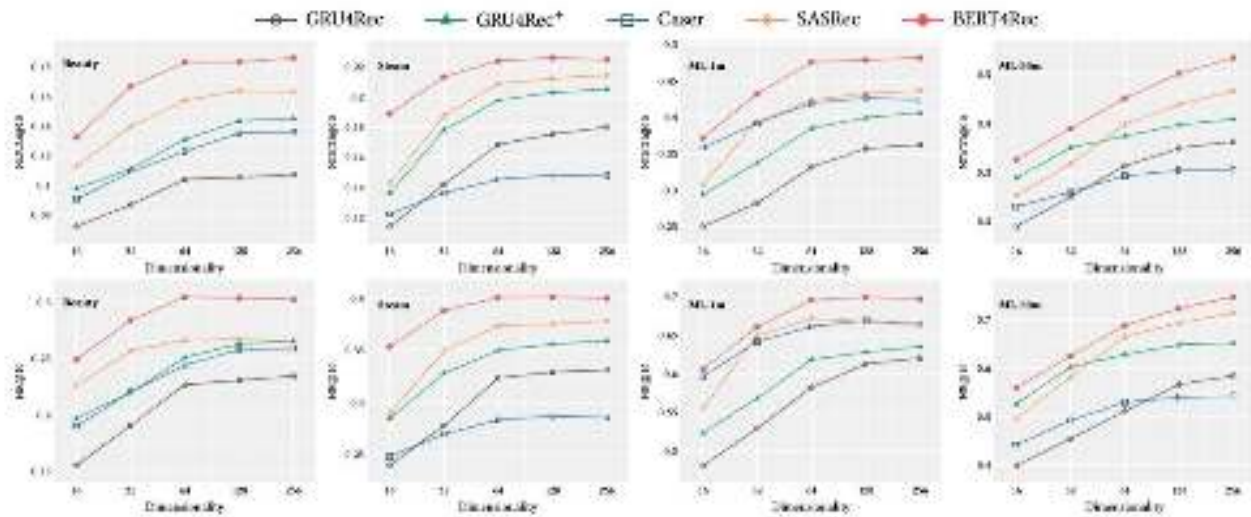


Figure 3: Effect of the hidden dimensionality d on HR@10 and NDCG@10 for neural sequential models.

- A general conclusion regarding mask proportion is that performance decreases as mask proportion (p) increases (Figure 4 from (Sun, 2019)). Short sequence datasets (Steam and Amazon Beauty) offer the best performance at higher p and long sequence datasets (ML-1m and ML-20m) demonstrate the best performance at lower p .

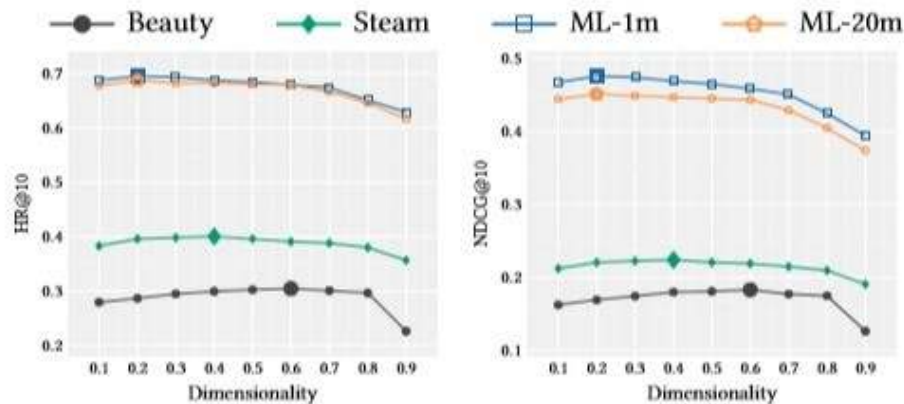


Figure 4: Performance with different mask proportion ρ on $d = 64$. Bold symbols denote the best scores in each line.

- Short sequence datasets prefer smaller maximum sequence length while long sequence datasets prefer larger sequence lengths. This means that recent items affect user's behavior more in short sequence datasets (as expected), while less recent items are more relevant in long sequence datasets.

Works Cited

Aggarwal, C. (2016). *Recommender Systems*. Springer.

- Devlin, J. C. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171-4186). Minneapolis, Minnesota: Association for Computational Linguistics.
- Geron, A. (2019). *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. O'Reilly.
- Hendrycks, D. G. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.
- Hidasi B., K. A. (2015). Session-based Recommendations with Recurrent Neural Networks. *ICLR*.
- Kang W.C., M. J. (2018). Self-Attentive Sequential Recommendation. *ICDM '18*.
- Ouyang Y., L. W. (2014). Autoencoder-Based Collaborative Filtering. *International Conference on Neural Information Processing* (pp. 284-291). Springer.
- Rendle S., F. C.-T. (2012). BPR: Bayesian Personalized Ranking from Implicit Feedback. *UAI '09 Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, (pp. 452-61). Montreal.
- Salakhutdinov, R. H. (2007). Restricted Boltzmann machines for collaborative filtering. *ICML '07 Proceedings of the 24th international conference on Machine learning* (pp. 791-798). Corvalis, Oregon: ACM.
- Sun, F. L. (2019). BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. *arXiv preprint arXiv:1904.06690*.
- Taylor, W. (1953). Cloze Procedure: A New Tool for Measuring Readability. *Journalism Bulletin*, 415–433.
- Vaswani, A. S. (2017). Attention is all you need. *Advances in neural information processing systems*, 5998-6008.
- Zhang S., Y. L. (2019). *Deep Learning based Recommender System: A Survey and New Perspectives*. *arXiv preprint arXiv:1707.07435*

Breakout 3

Title of the paper: Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology

Team members: Chris Alert, Geoffrey Hunter, Lisa Pritchett, Shahrzad Hosseini

Contributions: Chris Alert: summarized the overview of Q-learning, the proposed SlateQ algorithm, the methodology to adapt existing recommenders, Youtube experiments , conclusions and future directions, Lisa Pritchett: outlined the structure of the post

1. Introduction: Why should you read this?

Today, several companies deploy recommender systems in production that focus on short-term, "myopic" prediction of a user's immediate response to a recommendation. These optimize for first order effects: clickthrough, purchase, media consumption, but the algorithms used often ignore the second-order "feedback-loop" effects of their recommendations on longer term user behaviour. Take for example, recommend for a news article curator. Almost by definition, "fake news" articles have grab attention and likely attract users to click-through. However, reading such articles could erode the user's trust in the curator's credibility or influence the user's tastes for future content. Hence, recommending a "fake news" article could influence long-term value to the curator more than if they recommended an alternative "real news" article that had less immediate likelihood of clickthrough but a richer experience for the user.

Key Contributions

The paper set out to introduce three major insights to the reader:

- SLATEQ: a method that adapts value-based temporal difference and Q-learning algorithms to efficiently learn to recommend slates. It does so by estimating slate long-term value (LTV) as a function of its component item-wise LTVs then assembling optimal slates
- Describes a practical process whereby an existing system serving a "myopic" recommender system - that is, a system optimizing for short term, next item engagement (clickthrough/ purchase) - can be adapted to handle LTV (training, logging, serving)
- Demonstrates the success and validates the scalability of the decomposed TD-learning method using SLATEQ in simulation and in a live experiment on a small, but statistically significant segment of Youtube users

2. What is SARSA and Q-learning?

Markov Decision Process (MDP) Model for Slate Recommendation

In the paper, the authors set the problem up as a partially observable MDP to optimize degree of engagement over sessions. In this setting:

- user is presented a slate of recommended items

- user can select zero or more items from the slate
- each time an item is consumed, the user can return for additional slate recommendations or terminate the session

A user's response can be multi-dimensional, including: ratings/feedback or subsequent engagement with the content provider outside the recommendation system's direct control

Degree of engagement *an abstract notion of reward that encompasses any one or more of the possible user responses and the chosen metric*

The key components of the MDP are described below:

- S : user state - static user features (demographics), declared interests, dynamic user features (user context, etc), summarization of user history and past behaviour to capture aspects of the user's latent state
- A : actions
 - The set of all possible recommendation slates of size k
 - Recommendable items come from a fixed catalogue, I , so an action A is a set of k recommendable items
 - we assume every slate includes a $k+1$ st option: a null item that allows a user to select "no item" from the slate
- $R(s, A)$: reward function/ expected reward of a slate accounting for uncertainty in user response (degree of engagement)
- P : transition kernel
 - transition probability $P(s'|s, A)$ reflects the probability that a user transitions to state s' when action A is taken at user state s . This accounts for uncertainty in user response but also in the future context/ environment state. The largest influence on transition probability is the user choice model that specifies likelihood a user will pick recommended item a in A
- discount factor: between 0 and 1
- stationary, deterministic policy (π): $S \rightarrow A$ dictates the action to be taken at any state s

The goal is to find an optimal slate recommendation, A as a function of the state. To do so, we learn the value function of a fixed policy π .

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P(s' | s, \pi(s)) V^\pi(s'); \quad \text{where } \gamma \in [0, 1]$$

$$\text{The optimal policy gives: } V^*(s) = \max_{A \in \mathcal{A}} R(s, A) + \gamma \sum_{s' \in S} P(s' | s, A) V^*(s'); \quad \text{where } \gamma \in [0, 1]$$

Essentially, the value of a policy applied in a state, s , is a function of the estimated reward generated by that policy in state s , plus the discounted expected value of acting according to policy π over all possible future states transitioned to, s' . To be more concrete, if you take an action, A , at state s , the corresponding action value, or Q -function has value:

$$Q^\pi(s, A) = R(s, A) + \gamma \sum_{s' \in S} P(s' | s, A) V^\pi(s')$$

Similarly, it is the estimated reward for taking action A in state s , added to the discounted expected value of acting according to policy π over all the possible states s' the user could

transition to. Hence we weigh the likelihood of transitioning to candidate states s' given you take action A against the estimated value achievable in that state by following π .

With access to the reward and transition models, we can compute optimal policies $\pi^*(s)$ that produce optimal (max) Q value functions $V^*(s)$ and $Q^*(s,A)$.

SARSA is an on-policy learning method that leverages sampled data to learn/ approximate optimal policies. It uses training data of form (s, A, r, s', A') representing observed transitions and rewards generated by some policy given π . The $Q\pi$ function can be estimated based on iteratively acting greedily according to policy π . The update rule moves the estimate of the value at state s , based on a learning rate and the difference between: 1) actual value realized at the state by taking action A plus a discounted value realized at the future state by taking action A' and 2) the prior estimate of the value of taking action A at the current state.

$$\bar{Q}^\pi(s, i) = \bar{Q}^\pi(s, i) + \alpha[r + \gamma \sum_{j \in A'} P(j | s', A) \bar{Q}^\pi(s', j) - \bar{Q}^\pi(s, i)]$$

Q - learning is an off-policy learning method that directly estimates the optimal Q -function. This requires estimation of optimal slates given a state s - independent of data generating policy π - at training time. The Q -function approximator is then similarly updated in an iterative process comparing how well the realized reward in the future state compared to the function estimate for the state value.

$$\bar{Q}(s, i) = \bar{Q}(s, i) + \alpha[r + \gamma \max_{A \in \mathcal{A}} \sum_{j \in A'} P(j | s', A) \bar{Q}(s', j) - \bar{Q}(s, i)]$$

3. What is SlateQ Decomposition?

SLATEQ is a model that allows the Q -value of a slate to be decomposed into a combination of the item-wise Q -values of its constituent items. Three big challenges in the slate recommendation problem are:

1. Exploration: the huge number of ways to choose slate items and order them results in a huge action space that renders sufficient exploration impractical
2. Generalization: challenging across slates unless there is some compressed representation
3. Optimization: the combinatorial optimization problem of finding a slate with maximum Q -value must be solved is required for policy improvement. However, without assumptions / approximations the optimization would be unable to meet the latency requirements of a real production system

The SLATEQ decomposition allows both temporal difference (TD) learning algorithms like SARSA and Q -learning to learn LTVs at the individual item level despite the fact items are presented to users in slates. It then proposes different candidate methods (top-k, greedy and

reduction to a linear program [LP]) for in a final optimization step to efficiently (polynomial time) build slates to maximize LTV at training time and select optimal slates at serving time.

The benefits gained hinge on a few assumptions that seem reasonable for many real world recommendation settings and are crucial to simplifying the problem:

- **Single choice (SC):** A user consumes a single item from each slate (which may be the null item).
- **Reward/transition dependence on selection (RTDS):** the realized reward (user engagement) depends only on the item i in A consumed by the user. Similarly, the state transition $P(s' | s, A)$ depends only on the consumed i in A .

SLATEQ decomposition works with any choice model satisfying the two assumptions, though the optimization form may vary based on choice model. The decomposition of on-policy Q -functions for a fixed data generating policy π relies on an item-wise auxiliary function estimating the long-term value of consuming item i :

$$\bar{Q}^\pi(s, i) = R(s, i) + \gamma \sum_{s' \in S} P(s' | s, i) V^\pi(s'), \text{ for some item } i \in A$$

The paper proves a small proposition that then shows we can decompose the slate Q -values into Q -values for individual items. This accounts for the combinatorial action space challenge. A further proposition shows, conveniently, that while SLATEQ learns item-wise Q -values, it can be shown to converge to the correct slate Q -values under standard assumptions when assuming a known user choice model. This takes away reliance on sampled user choices and allows the algorithm to learn over a distribution of expected user behaviour. Ultimately this allows the algorithm to converge to the optimal Q -function and generalize over slates.

4. How to find the *slate* with maximum long-term value?

Three methods are proposed for taking the output slate decomposition and optimizing for LTV: LP-based exact optimization, top-k method, greedy method.

The exact optimization method works for any general conditional choice model of the form:

$$P(j | A) = \frac{v(x_{\{j\}})}{\sum_{i \in A} v(x_{\{i\}})}$$

The conditional logit model used in the paper assumes the user in state s selects item i with unnormalized probability v :

$$v(s, i) = e^{\tau u(s, i)}, \text{ for item } i \in A$$

In this scenario, the LP is maximizing the expected decomposed Q -value given a user selects an item recommended from a slate of a fixed size (k). The direct problem is a fractional mixed

integer problem so the authors propose a reducible problem whose optimal solution yields an optimal slate in the original fractional MIP. Here the x_i 's indicate whether the item i occurs in slate A .

$$\max \sum_{i \in \mathcal{I}} \frac{x_i v(s, i) \bar{Q}^\pi(s, i)}{v(s, \perp) + \sum_j x_j v(s, j)}$$

$$\text{s.t. } \sum_{i \in \mathcal{I}} x_i = k; x_i \in \{0, 1\}, \forall i \in \mathcal{I}$$



$$\max \sum_i y_i v(s, i) \bar{Q}^\pi(s, i)$$

$$\text{s.t. } v(s, \perp) + \sum_i y_i v(s, i) = 1$$

$$t \geq 0; \sum_i y_i \leq k$$

The LP relaxation is also solvable in polynomial time in the number of items (assuming a fixed slate size k). This involves a relaxation of the binary indicators [Chen and Hausman 2000] and further transformation [Charnes-Cooper 1962] to recast as a non-fractional LP. The authors introduce additional variable t for the inverse choice weight of the selected items and auxiliary variables y_i .

$$t = \frac{1}{v(s, \perp) + \sum_j x_j v(s, j)}; y_i = \frac{x_i}{v(s, \perp) + \sum_j x_j v(s, j)}$$

The optimal solution to the LP yields the optimal x_i assignment in the fractional LP which ultimately gives the optimal slate and can be solved in polynomial time. The optimization is also helped since in production many recommender systems first filter the item catalogue to a shortlist before choosing a slate at serving time.

Other methods proposed for constructing the slate of k items included:

top-k: take the top k values sorted based on $v(s, i) \bar{Q}(s, i)$

Greedy: take the top k one-by-one based on maximal marginal contribution

$$\arg \max_{i \notin A'} \frac{v(s, i) \bar{Q}(s, i) + \sum_{\ell \in L} v(s, i_{(\ell)}) \bar{Q}(s, i_{(\ell)})}{v(s, i) + v(s, \perp) + \sum_{\ell \in L} v(s, i_{(\ell)})}$$

These heuristic approaches give no theoretical guarantees. The approximation ratio (relative to optimal solution) for top-k can even be shown to be unbounded but in the experiments performed reasonably well.

These optimization options yield various algorithm variants, based on the optimization technique used and the learning method (SARSA/ Q-learning or non-RL (myopic) models). While training the Q-learning algorithm, slate optimization is required and we may use any of the three optimization methods.

Method shorthand	Description
GS / GT	Greedy serving / training
TS / TT	Top-k serving / training
OS / OT	Optimal serving / training

Q-learning Algorithm Variants

Training	Serving			
	Top-k Greedy LP (Opt)	Top-k	Greedy	LP (Opt)
		QL-TT-TS	QL-TT-GS	QL-TT-OS
		QL-GT-TS	QL-GT-GS	QL-GT-OS
		QL-OT-TS	QL-OT-GS	QL-OT-OS

SARSA and Myopic recommender

Serving	SARSA	Myopic
Top-k	SARSA-TS	MYOP-TS
Greedy	SARSA-GS	MYOP-GS
LP (Opt)	SARSA-OS	MYOP-OS

In the results section, these variants were compared against recommending random slates as well as against full Q-learning (for a tractably small k).

Adapting Myopic Systems into Long-Term Systems

A key contribution of this paper was the proposed methodology for extending an existing myopic recommender to account for long term behaviour based on Q-values. The core components needed in the foundational myopic system were: *logging* of user impressions and feedback, *training* of some model to predict user responses for user-item pairs (subsequently aggregated by some scoring function) and lastly *serving* recommendations, ranking items by score.

State Space

An ideal state space in an MDP is comprised of a feature set that is rich enough to encode most relevant historical context and that is at least approximately predictive of the next state/ user's response (and hence reward signal). The feature engineering that goes into building a production-grade baseline recommendation system typically satisfies these necessary criteria. Hence, the RL or TD learning models can leverage the same logged data processed through the same feature engineering pipelines.

Generalization across users

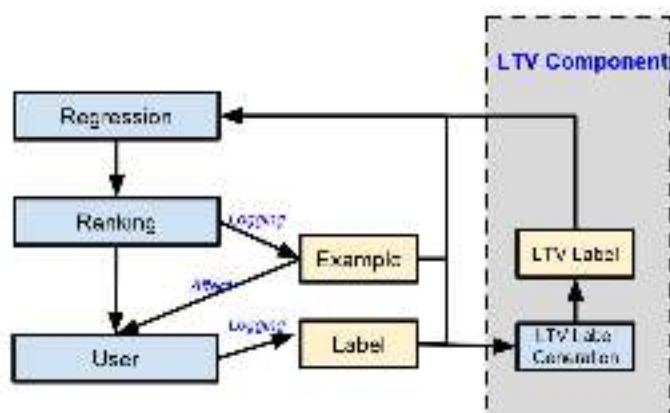
If we treat each user as a distinct environment/MDP, few would have enough interactions to train the algorithms. However, featurization used in myopic recommenders represent users in terms of general characteristics observable across the full userbase. Hence, learning a Q-function based on this representation SLATEQ can learn generalizable user dynamics.

User Response Modeling

In place of the unobservable $v(s,i)$ [probability a user engages with some item i in state s], the algorithm leverages the pre-trained and highly tuned existing myopic recommender system's pCTR predictions. You could further extend the existing model to build a multi-task model that incorporates long-term engagement prediction.

Infrastructure

The figure below shows the setup for SARSA learning, the method tested in YouTube's live experiment in production with the described SLATEQ algorithm.



Myopic: the regression model predicts immediate user response

LTV-based: label generation provides LTV labels allowing the regressor to model $Q^{\pi}(s,a)$

Training

Training happens in periodic batches, then the model is pushed to the server. The ranker uses the latest model to recommend items and logs user feedback to feed as new training data. The iterative model training with LTV labels is a form of generalized policy iteration. Each DNN model will represent the value of the policy that generated the prior batch of training data, so training is effectively policy evaluation. Policy improvement comes since the ranker acts greedily according to the value function.

LTV label generation

A main DNN learns the LTV of individual items, $Q^{\pi}(s,a)$ - extended from the myopic pCTR (probability of clickthrough or other engagement measure) model. LTV values (Q-values) are generated using a separate (fixed) label network $\bar{Q}_{label}(s,a)$. Then periodically the main (multi-task) network weights get copied to the label network. This helps to reduce the variability in the labels used to train the algorithm online.

$$\text{LTV label: } \bar{Q}_{label}(s,a) \text{ gets } \bar{Q}_{label}(s,a) + \bar{Q}_{myopic}(s,a) + \sum_{j' \in A'} v(s', j'|A') \bar{Q}_{label}(s', j')$$

Results

YouTube tested the SARSA-TS (greedy top-k when serving) with live users. The experiment was run on $O(10^9)$ users and $O(10^8)$ items in its corpus. The incumbent system is two part: first a candidate generator algorithm returns list of $O(100)$ items that best match the user context. Next, a ranker scores and ranks candidates using a DNN with both user context and item features as input and optimizing multiple objectives (clicks, expected engagement, and several other factors).

For the experiment, the myopic engagement measure is replaced by an LTV estimate in the ranker scoring function. They retained other predictions and incorporate them into candidate scoring like in the myopic model. The objective: maximizing cumulative expected engagement with user trajectories capped at N days. Since users visits can be arbitrarily spaced in time, discounting is done on a time basis to handle credit assignment over large time gaps. Otherwise, the setup adapted the myopic setting to LTV just as previously described.

Network architecture

Layers	4 hidden layers of sizes 2048, 1024, 512, 256
Activation function	RELU activation functions on each of the hidden layers
Output head(s)	LTV/Q-value (see above equation), pCTR, other user responses matching the production incumbent myopic model
State	User features (past history, behaviour, responses, static user attributes)
Framework	Tensorflow distributed [Abadi et al., 2015]
Training scheme	On-policy over pairs of consecutive start page visits, with LTV labels computed as above and using top-k optimization for serving (and ordering at serving time*)

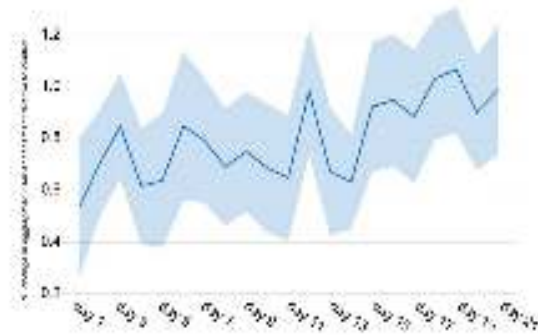


Figure 2: Increase in user engagement over the baseline. Data points are statistically significant and within 95% confidence intervals.

Over the 3 week experiment period, the SLATEQ SARSA-TS algorithm served a small but statistically significant group of users (exact number not mentioned in paper). It consistently and significantly outperformed the highly tuned incumbent myopic model on the key metric under consideration.

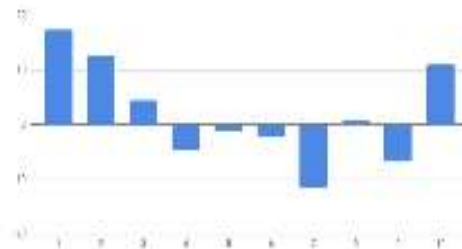


Figure 3: Percentage change in long-term user engagement vs. control (y-axis) across positions in the slate (x-axis). Top 3 positions account for approximately 95% of engagement.

The results also showed that users in the experient group had higher LTV from items ranked higher in the slate compared to users in the control group. This was particularly important since the slate size varied depending on the user's scrolling. Even in this setting, top-k slate optimization performed well.

Future Directions

One proposed direction was to relax some of the assumptions regarding the interaction between user choice and system dynamics. For example, allowing unconsumed items presented in a slate to influence a user's latent state or choice models allowing multiple items to be consumed. Another direction is to explore alternative user choice models beyond the general conditional choice model and even explore learning/ estimating the user choice model parameters while learning the Q-function.

The last mentioned direction which would be hugely beneficial to the recommender system research community as a whole was potentially open sourcing their RecSim simulation environment.

References:

[Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology](#), Le , Jian, Wang et al, 2019 (the paper)

[Reinforcement Learning: An Introduction](#), Sutton and Barto 2017 Second Edition draft

Sutton and Barto 1998, Mnih et al, 2015

Breakout 4

Title of the paper: Deep Learning Recommendation Model for Personalization and Recommendation Systems

Team members: Jiri Stodulka, Andrey Melnik, Lediona Nishani, Doug Rangel, Kristopher Makuch [JS, AM, LN, DR, KM]

Contributions:

In the process...

Abstract & Intro

- Deep neural networks applied to recommendation models have to handle categorical features. Such neural networks have different architecture and are not well studied.
- The authors develop a deep learning recommendation model and implement in PyTorch and Caffe2 frameworks.
- Parallelism.... [AM: I did not get it yet]

The authors present a model ("DLRM", deep learning recommendation model), in which they combine two approaches to recommender systems:

1. Classical recommendation systems view, which focuses on similarities between users and products and includes neighbourhood methods and latent factor methods.
2. Predictive analytics view, which employs statistical models to make predictions based on given data.

Model Design and architecture

References

1. <https://arxiv.org/abs/1906.03109v2> [The paper]
2. <https://ai.facebook.com/blog/dlrm-an-advanced-open-source-deep-learning-recommendation-model/> [This blogpost has a nicer diagram outlining the model]

To be continued...

Breakout 5

Title of the paper: Large-scale Parallel Collaborative Filtering for the Netflix Prize

Team members: Elham Karami, Raul Morales Delgado, Alireza Darbehani

Contributions: Raul Morales Delgado wrote the summary for sections 1, 2, 3.1. Elham Karami wrote the summary for sections 3.2, 4. Alireza Darbehani wrote the summary for sections 5, 6.

1. Summary

This paper presents a recommendation system technique based on collaborative filtering. Specifically, it proposes and describes the use of the *Alternating-Least-Squares with Weighted- λ -Regularization* (ALS-WR) parallel algorithm, which was designed for the Netflix Prize. The results achieved with ALS-WR include a model with 1,000 hidden features that obtained a RMSE score of 0.8950 — top of the class for a pure method — and yielded a total improvement of 5.91% over Netflix's CineMatch recommendation system.

2. Introduction

The collaborative filtering approach for recommendation systems is based on using the aggregated behavior or taste of a large number of users over a large set of relevant items. In this sense, the main objective of the system is to predict the rating that user i will give to an item j , which translates into minimizing the root mean squared error (RMSE) between the predicted value and its target.

However, an algorithm that tries to optimize a loss function over a matrix that includes all users and their respective interaction with all items turns out to be computationally expensive — the Netflix Prize proposes a matrix with 100 million ratings (~480,000 users and ~18,000 movies). Given the sparsity of the matrix that was proposed by Netflix, the authors identified a number of problems arose:

1. The size of the matrix — by the time this paper was presented — was 100 times larger than any previous benchmark datasets;
2. Only 1% of the matrix contained observations, the rest of them were missing;
3. Noise existed in both training and test sets, due to the difficulty of predicting human behavior; and,
4. The distributions between sets were different — the training set contains data from 1995–2005, while the test set contains only data from 2006.

Finally, it is important to mention that Netflix's recommendation system had a RMSE score of 0.9514 on the test set, and the objective of the prize was to reduce this score by 10%.

To solve this problem, Zhou et al. presented and described the ALS-WR parallel algorithm, which they ran on Matlab on a Linux cluster, where they achieved a 5.91% improvement on the RMSE score.

As it will be seen in the following sections, the main purpose of ALS-WR is to deal with ill-posed problems and sparse matrices without the risk of overfitting and with models that have a large number of hidden features.

3. Problem Formulation and Approach Analysis

This section covers section 2 and subsection 3.1 of the original paper. Variables and indices have been conserved from those used in the paper unless specified.

3.1. Problem Formulation

Given a user-movie matrix with n_u number of users and n_m number of movies, the resulting matrix can be denoted by

$$\mathbf{R} = \{r_{ij}\}_{n_u \times n_m},$$

where r_{ij} represents the rating score of movie j by user i , whether it exists — as a real number — or not — it is missing. In such case, the objective of a recommendation system would be to estimate some of the missing ratings in $\mathbf{R}$.

In order to predict these estimates, it is necessary to work with low-rank matrices — two low-rank matrices that would model both the users and the movies through a set of features, and whose multiplication would yield an approximation of r_{ij} . Let us define the user feature matrix $\mathbf{U}$ and the movie feature matrix $\mathbf{M}$ as

$$\mathbf{U} = \{u_{ki}\}_{n_f \times n_u}, \quad \text{forall } i = 1 \dots n_u, k = 1 \dots n_f;$$

$$\mathbf{M} = \{m_{kj}\}_{n_f \times n_m}, \quad \text{forall } j = 1 \dots n_m, k = 1 \dots n_f;$$

where n_f is the dimension of feature space — the number of hidden features that will be considered for each of the above matrices. In other words, column vectors $\mathbf{u}_i$ and $\mathbf{m}_j$ represent the hidden features for user i and movie j , respectively, and are a subset of $\mathbf{R}^{n_f}$.

Assuming that the ratings are fully predictable and that n_f is sufficiently large, then $r_{ij} = \mathbf{u}_i^T \mathbf{m}_j$ can be expected. But to obtain both $\mathbf{U}$ and $\mathbf{M}$, a loss function $\mathcal{L}$ has to be minimized first. In the paper, the mean-square loss function is used for that task. For a single rating:

$$\mathcal{L}^2(r, \mathbf{u}, \mathbf{m}) = (r - \langle \mathbf{u}, \mathbf{m} \rangle)^2.$$

From this expression, an empirical total loss function can be defined as the summation of loss for all ratings:

$$\mathcal{L}^{\text{emp}}(\mathbf{R}, \mathbf{U}, \mathbf{M}) = \frac{1}{n} \sum_{(i, j) \in I} \mathcal{L}^2(r_{ij}, \mathbf{u}_i, \mathbf{m}_j),$$

where n is the size of I , and I is the index set of known ratings.

Therefore, the minimization problem can be formulated as:

$$(\mathbf{U}, \mathbf{M}) = \underset{(\mathbf{U}, \mathbf{M})}{\text{arg}} \min \mathcal{L}^{\text{emp}}(\mathbf{R}, \mathbf{U}, \mathbf{M})$$

By formulating the problem like this, there will be only $(n_u + n_m) \cdot n_f$ free parameters to solve for — the index set I contains far less user-movie ratings than $n_u n_m$ elements. According to the paper, the dataset provided by Netflix only had 1.1% of known ratings.

The next problem that arises is related to working with such sparse matrix. Specifically, when solving for a relatively large n_f , the resulting feature matrices will overfit the data. To avoid overfitting, a Tikhonov regularization term is appended to $\mathcal{L}^{\text{emp}}$:

$$\mathcal{L}^{\text{reg}}_{\lambda}(\mathbf{R}, \mathbf{U}, \mathbf{M}) = \mathcal{L}^{\text{emp}}(\mathbf{R}, \mathbf{U}, \mathbf{M}) + \lambda (\|\mathbf{U}\|_{\Gamma_U}^2 + \|\mathbf{M}\|_{\Gamma_M}^2),$$

where Γ_U and Γ_M are suitable Tikhonov matrices, which will be discussed in the next subsection.

3.2. ALS with Weighted-Regularization Approach

Considering that matrix $\mathbf{R}$ contains signals and noise, it is necessary to remove the noise such that the recovered signals can be used to predict missing ratings. As stated in the paper, a common approach for matrix factorization is using *Singular Value Decomposition*, SVD, where an approximation to the original matrix $\mathbf{R}$ is achieved by finding two rank- k matrices such that $\tilde{\mathbf{R}} = \mathbf{U}^T \mathbf{M}$. SVD provides a solution that minimizes the Frobenious norm of $\mathbf{R} - \tilde{\mathbf{R}}$.

However, regularized least squares (RLS) methods are preferred when a problem is ill-posed — the number of variables surpasses the number of observations in the dataset — or when, even if the number of observations is larger than the number of variables, the resulting learned model overfits to the dataset. According to Zhou et al., both problems occurred when SVD was applied to the Netflix dataset, so the authors proposed the use of Tikhonov regularization — also known as Ridge regression — to overcome them. Furthermore, they indicate that the inherent sparsity of the user-movie matrix makes it not possible for standard SVD algorithms to find $\mathbf{U}$ and $\mathbf{M}$.

To solve the low-rank matrix factorization problem, the paper proposes the use of *alternating least squares* (ALS) in the following way:

1. Initializing matrix $\mathbf{M}$ by assigning the average rating of each movie as the first row and small random numbers for the remaining ones;
2. Holding $\mathbf{M}$ constant and solving the lost function minimization for $\mathbf{U}$ — the sum of squared errors, $\mathcal{L}^{\text{reg}}_{\lambda}(\mathbf{R}, \mathbf{U}, \mathbf{M})$;
3. Holding $\mathbf{U}$ constant and solving the lost function minimization for $\mathbf{M}$;
- and,
4. Repeating steps 2 and 3 until a stop criterion is satisfied — which for the proposed model was 0.0001, or 1 bps.

As stated in the previous subsection, the authors deemed necessary the use of Tikhonov regularization, which penalizes large parameters, given that ALS on its own would overfit to the dataset. The Tikhonov matrices they chose for the loss function are stated in the following expression:

$$f(\mathbf{U}, \mathbf{M}) = \sum_{(i, j) \in I} (r_{ij} - \mathbf{u}_i^T \mathbf{m}_j)^2 + \lambda \left(\sum_i n_{u_i} \|\mathbf{u}_i\|^2 + \sum_j n_{m_j} \|\mathbf{m}_j\|^2 \right),$$

where n_{u_i} and n_{m_j} denote the number of ratings provided by user i and the number of ratings that movie j has, respectively. Similarly, n_{u_i} and n_{m_j} are the cardinality of I_i and I_j , respectively, where I_i represents the set of movies that user i rated, and I_j represent the set of users that rated movie j . Finally, the authors define the Tikhonov matrices as $\Gamma_{\mathbf{U}} = \text{diag}(n_{u_i})$ and $\Gamma_{\mathbf{M}} = \text{diag}(n_{m_j})$ — they will be scalars, $\Gamma_{\mathbf{U}}$,

$\gamma_{\mathbf{M}} \in \mathbb{R}^1$, that will penalize the loss function depending on the number of ratings that each user i or movie j have.

After $\mathbf{M}$ is initialized as stated above, in step 1, the following step is to solve for $\mathbf{U}$ holding $\mathbf{M}$ constant. $\mathbf{U}$ will be solved per user — column by column. In other words, u_i will be determined using the regularized least squares loss function provided above, and it will involve the existing ratings by user i , and all the hidden feature movie column vectors, $\mathbf{m}_j$, that user i has rated. The derivation process is shown in the following lines.

Considering the ALS problem, minimizing the loss function can be achieved as follows:

$$\begin{aligned} \frac{\partial}{\partial u_{ki}} \frac{\partial}{\partial u_{ki}} f &= 0, \quad \forall i, k \\ \implies \frac{\partial}{\partial u_{ki}} \left(\sum_{(i,j) \in I} (r_{ij} - \mathbf{u}_i^T \mathbf{m}_j)^2 + \lambda \sum_i \|\mathbf{u}_i\|^2 + \lambda \sum_j \|\mathbf{m}_j\|^2 \right) &= 0, \quad \forall i, k \end{aligned}$$

Given the column vectors,

$$\begin{aligned} \mathbf{u}_i &= \begin{bmatrix} u_{i1} & \dots & u_{ki} & \dots & u_{in_i} \end{bmatrix} \\ \mathbf{m}_j &= \begin{bmatrix} m_{1j} & \dots & m_{kj} & \dots & m_{n_jj} \end{bmatrix} \end{aligned}$$

a partial derivation over u_{ki} will eliminate all variables that do not contain it, and will modify the index set I such that now it only contains the set of movies j rated by user i , $j \in I_i$. Therefore,

$$\begin{aligned} \frac{\partial}{\partial u_{ki}} \left(\sum_{(i,j) \in I} (r_{ij} - \mathbf{u}_i^T \mathbf{m}_j)^2 + \lambda \sum_i \|\mathbf{u}_i\|^2 \right) &= 0, \quad \forall i, k \\ \implies \sum_{j \in I_i} \left((r_{ij} - \mathbf{u}_i^T \mathbf{m}_j) \mathbf{m}_j \right) &+ \frac{\partial}{\partial u_{ki}} \left(\sum_i \|\mathbf{u}_i\|^2 \right) = 0, \quad \forall i, k \end{aligned}$$

\$\$

$$\implies \sum_{\substack{j \in I_i}} \left(\mathbf{u}_i^T \mathbf{m}_j - r_{ij} \right) \cdot \frac{\partial}{\partial u_{ki}} \left(u_{1j} m_{1j} + \dots u_{kj} m_{kj} + \dots u_{n_j} m_{n_j} \right) + \frac{1}{2} \lambda \frac{\partial}{\partial u_{ki}} \sum_i n_{u_i} u_{ki}^2 = 0, \\ \quad \forall i, k$$

\$\$

\$\$

$$\implies \sum_{\substack{j \in I_i}} \left(\mathbf{u}_i^T \mathbf{m}_j - r_{ij} \right) \cdot m_{kj} + \lambda n_{u_i} u_{ki} = 0, \quad \forall i, k$$

\$\$

And rearranging the last expression above,

\$\$

$$\sum_{\substack{j \in I_i}} m_{kj} \cdot \mathbf{m}_j^T \mathbf{u}_i + \lambda n_{u_i} u_{ki} = \sum_{\substack{j \in I_i}} m_{kj} \cdot r_{ij}, \quad \forall i, k$$

\$\$

Finally, doing the summations for all \$k\$,

\$\$

$$\mathbf{M}_{I_i} \mathbf{M}_{I_i}^T \mathbf{u}_i + \lambda n_{u_i} \mathbf{u}_i = \mathbf{M}_{I_i} \mathbf{R}_{(i, I_i)}^T, \quad \forall i$$

\$\$

And factorizing \$\mathbf{u}_i\$ from the left side of the previous expression:

\$\$

$$\left(\mathbf{M}_{I_i} \mathbf{M}_{I_i}^T + \lambda n_{u_i} \mathbf{E} \right) \mathbf{u}_i = \mathbf{M}_{I_i} \mathbf{R}_{(i, I_i)}^T, \quad \forall i$$

\$\$

Where \$\mathbf{M}_{I_i}\$ is a sub-matrix of \$\mathbf{M}\$, \$\mathbf{M}_{I_i} \in \mathbb{R}^{n_f \times I_i}\$, \$\mathbf{E}\$ is an identity matrix such that \$\mathbf{E} \in \mathbb{R}^{n_f \times n_f}\$, and \$\mathbf{R}_{(i, I_i)}\$ is the \$i_{th}\$ row vector of \$\mathbf{R}\$ with columns \$j \in I_i\$. If the terms are grouped such that,

\$\$

$$\mathbf{A}_i = \mathbf{M}_{I_i} \mathbf{M}_{I_i}^T + \lambda n_{u_i} \mathbf{E}; \quad \mathbf{V}_i = \mathbf{M}_{I_i} \mathbf{R}_{(i, I_i)}^T,$$

\$\$

then the previous expression can be simplified to:

\$\$

$$\mathbf{u}_i = \mathbf{A}_i^{-1} \mathbf{V}_i, \quad \forall i.$$

\$\$

After this step (step 2) is completed, then step 3 can be executed by holding $\mathbf{U}$ constant and solving for $\mathbf{M}$. Using the same procedure as above will yield:

\$\$

$$\mathbf{m}_j = \mathbf{A}_j^{-1} \mathbf{V}_j, \quad \text{forall } j,$$

\$\$

where $\mathbf{A}_j = \mathbf{U}_{I_j} \mathbf{U}_{I_j}^T + \lambda \mathbf{n}_j \mathbf{E}$ and $\mathbf{V}_j = \mathbf{U}_{I_j} \mathbf{R}_{(j, I_j)}$. Analogously, $\mathbf{U}_{I_j}$ is a sub-matrix of $\mathbf{U}$, $\mathbf{U}_{I_j} \in \mathbb{R}^{n_f \times I_j}$, and $\mathbf{R}_{(j, I_j)}$ is the j -th column vector of $\mathbf{R}$ with rows $j \in I_j$.

As shown in this subsection, each iteration of this algorithm will minimize the loss function $\mathcal{L}^{\text{reg}}_{\lambda}(\mathbf{R}, \mathbf{U}, \mathbf{M})$, which will consequently improve the estimates of the hidden features of $\mathbf{U}$ and $\mathbf{M}$, thus improving the model's approximation to $\mathbf{R}$.

3.3. Parallel ALS with Weighted- λ -Regularization

The parallelization of the alternating least square solution (ALS) has been implemented using the MATLAB software package. The latest version of the software allows for several separate MATLAB workspaces, which are running on different hardware platforms, to communicate with each other. Each copy of the software is called a "lab". The matrices used in the labs can be of three different types:

1. Private: each lab has its own copy, and their values differ.
2. Replicated: private, but with the same value on all labs.
3. Distributed: there is one matrix, but with rows, or columns, partitioned among the labs.

In this step, two distributed versions of the rating matrix $\mathbf{R}$ are used. One of them is distributed by rows (i.e., users) and the other by columns (i.e., by movies). To update $\mathbf{M}$ ($\mathbf{U}$), the columns (Rows) of $\mathbf{R}$ are divided into equal-size blocks and a replicated version of $\mathbf{U}$ ($\mathbf{M}$) is used for updating each block. The MATLAB code snippet used for updating one block of $\mathbf{M}$ is brought in the paper.

Since the proposed parallelization method uses a distributed algorithm, the only computation cost is the broadcast step, and it takes less than 5% of the total run time.

4. Performance for the Netflix Prize Problem

While the test dataset is hidden by Netflix, the training dataset consists of a probe dataset which has the same distribution as the test dataset. As such, the probe dataset is used to

optimize the model parameters. The optimal value of λ is obtained by trial and error, and the iterations for improving the RMSE are stopped when the improvement is less than 1 bps.

4.1. Post-processing

The authors propose two post_processing methods for improving the algorithms performance.

1. If the mean of the predictions (P) is not equal to the mean of the test set, it can be corrected for by shifting the mean by a constant which equals to: $\text{mean}(\text{test}) - \text{mean}(P)$
2. Various predictors can be combined linearly to obtain a better predictor.

The details are explained in the paper.

4.2. Experimental Results for ALS

The experimental results for finding the optimal values of λ and n_f (number of hidden features) are shown in figures 1 and 2, respectively. Figure 1 depicts that with fixed values of λ and n_f the convergence happens after about 20 iterations where the best performer is $\lambda = 0.065$. Figure 2 shows that with fixed λ , the RMSE monotonically decreases with larger n_f , even though the improvement diminishes gradually. The main finding of these experiments is that the proposed algorithm never overfits the data if the number of iterations or the number of hidden features are increased. A series of experiments with large values of n_f indicates that with $n_f = 1000$ an RMSE score of 0.8985 can be obtained which translates into a 5.56% improvement over Netflix's CineMatch, and it represents one of the top single-method performances.

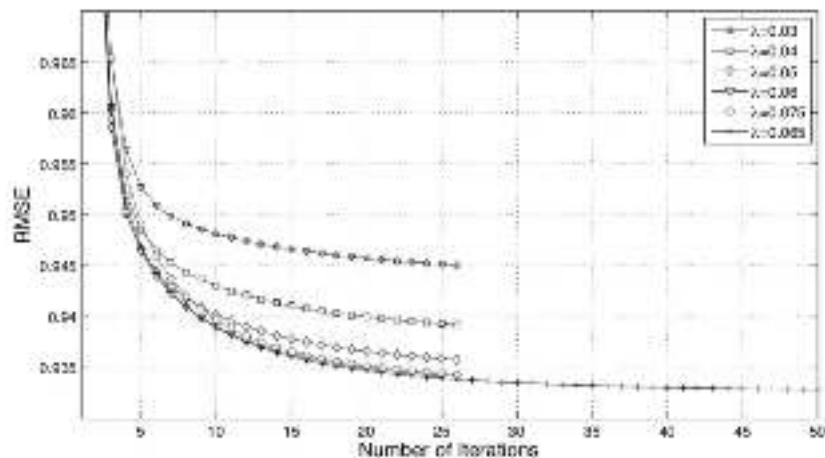


Fig. 1. Comparisons of different λ values for ALS-WR with $n_f = 8$. The best performer with 25 rounds is $\lambda = 0.065$. For this fixed λ , after 50 rounds, the RMSE score still improves but only less than 0.1 bps for each iteration afterwards.

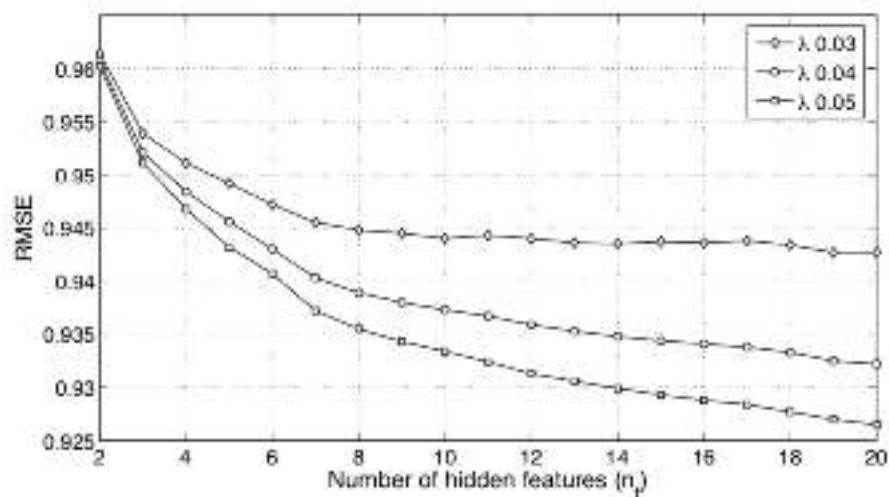


Fig. 2. Performance of ALS-WR with fixed λ and varying n_f .

4.3. Other Methods and Linear Blending

Two of the popular collaborative filtering methods include the Restricted Boltzmann Machine (RBM) and the k-nearest neighbor (kNN) method which are implemented and their performances are reported. The RBM is an NN-based method where there are visible states and hidden states, and undirected edges connecting each visible state to each hidden state. To improve performance, both methods are parallelized using MATLAB. For RBM itself, a score of 0.9181 is obtained. For kNN with $k = 21$ and a good similarity function, a RMSE of 0.9270 is obtained. Linear blending of ALS with kNN and RBM yields a RMSE of 0.8952 (ALS + kNN + RBM), and it represents a 5.91% improvement over Netflix's CineMatch system.

5 Related Work

There are similar researches on recommender systems and low-rank matrix approximations and the Netflix Prize in both academia and industry. Here we review a couple of similar researches on each topic:

5.1 Recommendation Systems

The recommender Systems research and products are divided into two categories at a high-level view, Content-Based, and Collaborative-Filtering. Collaborative Filtering methods use the behavior of a large set of users against a set of products or choices. In this method, the goal is to make an estimation of what item/product would be the next choice of a user based on the similarity of the user's previous choices and other users' choices. Some similar researches using Collaborative filtering are [GroupLens](#) and [Bellcore Video Recommender](#). There is also

some example of combining both CF and CB recommenders such as [Fab System](#) and [Unified Probabilistic Framework](#).

5.2 Netflix Prize Approaches

The paper [Restricted Boltzmann Machines for Collaborative Filtering](#) used the Restricted Boltzmann Machine to obtain a Root Mean Square error score of 0.91 for the Netflix Prize. They also used a Low-Rank Approximation approach which achieved a score of above 0.91 using 20-60 hidden features. They used an SVD function which does the same task as ALS-WR in this paper. However, this paper used 1000 features to improve the score of RMSE.

Another paper ([The BellKor Solution to the Netflix Grand Prize](#)) uses a neighborhood base technique that combines the k-nearest neighbor and low-rank approximation. They won the prize in 2007 with the RMSE score of 0.87.

5.3 Low-Rank Approximation

There are other matrix approximation methods are also proposed for the Netflix Prize Competition. The paper [Collaborative Filtering via Ensembles of Matrix Factorizations](#) proposed nonnegative matrix factorization and maximum margin matrix factorization as the Low-Rank Approximation methods for the Netflix Prize.

6 Concluding Remarks

This paper introduced a new method to use Collaborative Filtering on really large scale datasets such as the Netflix competition. Slightly better results could be achieved by tweaking the RBM and KNN implementation, however, the goal of this paper was to achieve a significant improvement on the run-time performance by providing parallelization.

Breakout 6

Title of the paper: Amazon's item-to-item collaborative filtering

Team members: Gurinder Ghotra

Contributions:

Amazon recommendation Paper-Item-to-item collaborative filtering

Some challenges facing Amazon recommendation algorithms

1. The authors of the paper worked on the challenges posed when working with big data for recommendation. The e-commerce giant's algorithms have many challenges to overcome. For example the huge amount of data, with tens of millions of customers and millions of distinct catalog items, scaling become a difficulty and is of paramount. Many applications need results to come in real-time with maximum time lag at half a second. Apart from these, while there is a lot of data about customers, however, new customers would have extremely limited information making it a challenge to have meaningful recommendations for them. The polar opposite of this could be that older customers can have a glut of information based on their extensive purchase history. The algorithm also needs to be sensitive to changes as Customer data is volatile. Each interaction provides valuable customer data and the algorithm must respond immediately to new info.

Metrics:

Metrics that they watch out for and every other email marketing and web based advertisement - the click through and conversion rates.

On these measures item-to-item CF has been very effective.

Other methods that the author considered to compare item-to-item collaborative filtering with

The authors cite 3 common approaches to solve the recommendation problem

- 1. Traditional collaboration filtering**
- 2. Cluster models**
- 3. Search based methods**

Most recommendation algos fall under 2 categories

- 1. Based on customers:**
 - a. here the algo starts by finding similar customers based on items searched and ratings given.**
 - b. The algo then aggregates items from similar customers**
 - c. Removes the items that a specific user has used and recommends the others**
- 2. Based on items:**
 - a. Herein are search based and item-to-item collaborative filtering look for similarity in items-- aggregates them and recommends them**

Traditional Collaborative Filtering issues

How does it work:

The algorithm creates an N-dimensional vector , where N is the distinct number of items in the catalog. The components of the vector are +ve for purchased or positively rated items and -ve for negatively rated items. Best selling items could dominate this vector based on its popularity and positive rating. The algorithm compensates for best selling items by multiplying the N-dimensional vector with the inverse-frequency (the inverse of the number of customers who have purchased or positively rated the item) . This leads to discovery of less well-known items becoming more relevant.

The algorithm then makes recommendation based on a few customers that are most similar to the customer. Distance measures like cosine similarity are used.

There are other methods to select items in this algo. For instance, for similar customers - ranking of items based on number of users purchasing or positively rating it is done. Recommendation is based on the top most items.

Issues for using traditional collaborative filtering for recommendation:

Generating recommendation based on collaborative filtering is computationally expensive. With M as number of customers and N as number of items, the worst case is $O(MN)$. However, since the average customer vector is extremely sparse the algorithm's performance is closer to $O(M+N)$. Even with this reduced figure it is quite problematic, as the customer base is over 10 million and the number of items are over 1 million. This leads to scaling issues and sever performance issues.

Scaling issues are tackled based on :

Since performance is, in practice, $O(M+N)$, one approach is to reduce M , the number of customers. Herein a random sample is taken and this reduced set of customers are used to generate the similarity. As a consequence of using a subset of users, the customers chosen would be less similar to the user as the pool of selection has been reduced.

The other way to tackle scaling is to reduce the number of items, N . Herein the extremes – very popular or highly unpopular items - are disregarded. These items will never be recommended and another fallout being customers who have only purchased these extreme range products will not receive any recommendation.

Another approach based on reducing the number of items, N , is to limit the items to a specific product category. The impact is recommendations would not be able to come from different categories which is not desirable.

Cluster Models issues

1. Here the approach is to convert the recommendation problem to a classification problem.
2. The idea is to generate user clusters offline and then based on online experience can recommend based on the user cluster
3. The cluster is based on some distance measure such as Euclidean distance.
4. On large data sets optimal clustering is very computationally expensive the data is based on greedy cluster generation. Herein, certain clusters are randomly selected and then each user based on the distance measure is classified into one of these groups. There are provisions for the clusters to be merged.
5. So users based on similarity to vectors that summarize a segment.
6. These algos scale better than traditional collaborative filtering as the computationally expensive portion is done offline and there is a fixed number of clusters to compare against.
7. Issue is that the recommendation quality is low, as effectively every one in a specific cluster will get the same recommendation, hence personalisation is hampered.
8. The solution for this is to build more and more fine-grained clusters, however, beyond a point if the clusters are very small the cluster model suffers from the same issue of scaling and time for processing as faced by traditional collaborative filtering.

Search-Based Methods issues

1. Here the recommendation problem is treated as a search for related items.
2. Based on the users purchased or positively reviewed items the algo searches for similar titles, key words , same genre , similar key words. For example a purchase of the Godfather could trigger a related search for crime drama titles, other movies directed by the director or other movies of the actors.
3. The algo works when the user has few purchases / ratings and can also scale well under this situation
4. However, when the user has thousands of data points then the query for search is based on a subset of the searchable data and the recommendation quality suffers.

item-to-item collaborative filtering

1. The algo matches each of the users purchased and rated items to similar items.
2. Then combines those similar items into a recommendation list
3. To determine the most-similar match for a given table, the algo builds a similar - table items that customer tend to purchase together.
4. The psuedo code

For each item in product catalog, I1

For each customer C who purchased I1

For each item I2 purchased by customer C

Record that a customer purchased I1 and I2

For each item I2

Compute the similarity between I1 and I2 (this could be done by using a distance measure such as cosine similarity)

5. This code is very computationally expensive with worst case as $O(N^2M)$ best cases at $O(NM)$. However, it can be done offline

6. For online recommendation, the algo finds similar items to only the items that the user has used or rated.

7. It aggregates the similar items, recommends the most popular or correlated items

8. The scalability is due to the expensive calculation being done offline and giving the similar item table and the online component is only dependent on the number of items the user has purchased or rated, thus even million of users / items do not have an impact on the speed. And since the recommendation is based on similar items it offers personalised recommendations maintaining high quality.

Code for item-to-item with demo data to show how it would look

Run the code for movie database with change in cosine formula to accommodate for ratings. Provide demo id on how the recommendations would look

Breakout 7

Title of the paper: Pre-training of Context-aware Item Representation for Next Basket Recommendation

Team members: Marnie, Nakul, Huiping, Lindsey, Sebastian

Pre-training of Context-aware Item Representation for Next Basket Recommendation

...

- **Background**

- ***Recommender systems*** are used by businesses to determine which products their customers are likely to purchase and to keep their customers engaged, and to understand the needs of a given population. In order to personalize the experience for their customers, the system finds patterns in each customer's purchasing history, and compares each customer to others with similar behaviour or tastes. The patterns are captured in a mathematical model which will predict items that each user is most likely to engage with or purchase. Items are typically ranked using Mean Average Precision (MAP) or Mean Reciprocal Rank (MRR).

Market basket analysis is a technique used to discover and model relationships between items that people commonly purchase together, in order to make predictions and find patterns of co-occurrence. The items that a customer buys in a single purchase, transaction, or basket, make up the ***itemset***. The goal is to predict the top K items to recommend

for a given user based on a personalized, ranked list. Other names include: association analysis or frequent itemset mining. The technique is used for: marketing, physical store layout and website layout, fraud, cross-selling.¹ It quickly becomes a combinatorics nightmare, if the inventory is large.

Next basket recommendation is an important part of *market basket analysis* which aims to recommend a set of items to a user.

The innovation in the paper Pre-training of Context-aware Item Representation for Next Basket Recommendation is to encode the transaction context that captures the purchaser's intent into an *item* representation, as opposed to a fixed-length static vector for the item represented by utilizing a lookup layer. The idea had its genesis in the context-aware model used in Natural Language Processing (NLP) called BERT (Bidirectional Encoder Representations from Transformers). The authors replace the word Bidirectional with Item, naming their model IERT. Like BERT, it is a transfer learning approach, using a pre-training phase and a fine-tuning phase. The authors claim to outperform state-of-the-art based on the Ta-Feng data set.

Existing methods:

Collaborative filtering (CF)

The goal of Collaborative filtering is to identify users with similar interests and taste and determine what items they liked, and find items similar to items purchased in the past.

In other words, by using the behaviour of one customer, it is possible to predict what a similar customer will purchase. The assumption of collaborative filtering models is that similarity determines what a user is most likely to purchase.

-

Memory based collaborative filtering

Using cosine similarity, find similar users, or pearson correlation and take the weighted average of ratings. There is no training or optimization involved in memory based collaborative filtering.²

Model based collaborative filtering

Machine Learning is used to model user ratings of unrated items. Common methods: 1. Clustering algorithms like KNN; 2. Matrix Factorization like SVD; 3. Deep Neural Networks.²

Deep Neural Network (DNN) approaches:

HRM - Hierarchical Representation Model capture how the purchase of one item relates to the purchase of another item, sequential behaviours, and the purchaser's interests, general taste.

DREAM – Dynamic REcurrent bAsket Model builds on HRM, using RNNs to model the sequential relationships of itemsets.

ANAM- Attribute-Aware Neural Attention Model – also uses the attention architecture to adapt to a purchaser’s changing interests.

-

All of the deep learning approaches used prior to this paper model user-item interactions and item-item interactions, using a *fixed-length static vector representation of the items*. The proposal in this paper is to represent the user’s intentions with the item representation in a context-aware style, similar to BERT.

BERT

-

On October 11, 2018, a team from Google AI Language published a seminal paper that transformed (pun intended) Natural Language Processing NLP. Previously, bi-directional Long Short-Term Memory (BiLSTM) models, a specific implementation of the Recurrent Neural Network (RNN) architecture, were used to capture left-to-right and right-to-left context aware representations of words that were concatenated in “shallow” way. BERT was the first model to use deep bi-directional context-aware representation of words, disambiguating words, increasing the semantic representation captured from the surrounding text, without the use of RNNs. RNNs had significant limitations in capturing long term dependencies between words.

[more time and I will/can expand on this]

The inspiration for BERT came from a language comprehension and vocabulary test proposed by W. L Taylor in 1953, the Cloze task. The goal of the Cloze task is to fill in missing words from a passage of text.

There are two tasks in the *pre-training* phase of BERT:

1. Masked Language Model (MLM) -
15% of all WordPiece tokens in each sequence are randomly “masked” or hidden. The model is trained using cross entropy loss to predict the original token;
2. Next Sentence Prediction (NSP) -
A model is trained to predict the relationship between sentences from labelled sentence pairs. 50% of the time the label is *IsNext*, the other 50% are labelled *NotNext*.

Following the pre-training phase, BERT uses a *fine-tuning* approach or *feature-based* approach. The fine-tuning phase, adds a simple classification layer to the pre-trained model. The feature-based approach extracts fixed features from the pre-trained model and runs computationally inexpensive models using these representations.

Fundamentally, once you understand BERT, you can think of IERT as using the concept of a *word* in BERT as an *item*; and a *sentence* as a *basket*; and a *document* as a *sequence of baskets*. The similarities and differences of BERT and IERT are in the table below.

[continued on the next page]

BERT	IERT
Similarities	
<i>word</i>	<i>item</i>
<i>sentence</i>	<i>basket</i>
<i>document</i>	<i>sequence of baskets</i>
<i>MLM Masked Language Model</i>	<i>Masked Item Prediction</i>
<i>NSP Next Sentence Prediction</i>	<i>Next Basket Prediction</i>
Differences	

<i>word and sentence order embedded</i>	<i>order information among transactions embedded, item order within a transaction NOT embedded</i>
<i>Sentence pair relationship representation</i>	<i>intention-related transaction representations</i>
<i>Same word in different datasets has a similar meaning</i>	<i>Item ID not consistent across datasets</i>

Using the Attention mechanism, both local and global behaviours are captured. Local features are the from the current interaction. Global features are items-item or user-item over all interactions.

CONCLUSION

-

The NLP model BERT inspired context-aware item representation in a next basket recommendation model called IERT. The pre-training uses historical purchase transactions. The fine-tuning is done during the online recommendation. A next step would be to take some training data and attempt to replicate the results, which claims state-of-the-art performance. Given the possible combinations of items in a basket, it seems very ambitious for a very large inventory with diverse user populations to achieve realistic results. Unlike documents which have context, items purchased together can be very random. I remain sceptical!

1. <https://www.statisticshowto.datasciencecentral.com/market-basket-analysis/>
2. <https://towardsdatascience.com/various-implementations-of-collaborative-filtering-100385c6dfe0>